

Rapport d'expérimentation

Comparaison d'algorithmes de Deep Reinforcement Learning
sur quatre environnements de complexité croissante

Adam Yahia Abdchafee Ayman Tom

29 avril 2026

Contents

1	Introduction et objectifs	2
2	Les algorithmes : ce qu'on a implémenté et pourquoi ça marche (ou pas)	2
2.1	Q-Learning tabulaire	2
2.2	La famille DQN : trois problèmes, trois solutions	3
2.2.1	DQN	3
2.2.2	DDQN : résoudre la non-stationnarité des cibles	3
2.2.3	DDQN + Replay : décorréler les transitions	4
2.2.4	DDQN + PER : toutes les transitions ne se valent pas	4
2.3	REINFORCE et ses variantes de baseline	5
2.4	PPO	6
2.5	MCCR et MCTS : planifier sans apprendre	7
2.6	Expert Apprenti	7
3	Résultats globaux	8
4	Analyse par environnement	8
4.1	LineWorld et GridWorld : les révélateurs d'instabilité	8
4.2	TicTacToe : trois niveaux de performance	10
4.3	Quarto : le mur du signal sparse	11
5	Analyse du coût computationnel	12
6	Synthèse	12
6.1	Ce que les résultats confirment	12
6.2	Les surprises	13
6.3	Les limites	13
7	Conclusion	13

1 Introduction et objectifs

Dans ce projet, on a implémenté *from scratch* un ensemble d'algorithmes de deep reinforcement learning, sans utiliser de bibliothèques de haut niveau comme Stable-Baselines ou RLlib. L'objectif c'est de vraiment comprendre ce qui se passe à l'intérieur de chaque algorithme, et d'être capables d'expliquer les différences de performance qu'on observe expérimentalement.

On a couvert trois grandes familles :

- **Value-based** : Q-Learning tabulaire, DQN, DDQN, DDQN+Replay, DDQN+PER
- **Policy-gradient** : REINFORCE (3 variantes de baseline), PPO
- **Planification / Imitation** : MCRR, MCTS, Expert Apprenti

Tous les entraînements : **100 000 épisodes**, évaluation sur **1 000 épisodes** aux checkpoints 1k, 10k, 100k (seed = 42). Les métriques collectées sont le win rate, le loss rate, le draw rate, le nombre moyen de coups et le temps par coup.

2 Les algorithmes : ce qu'on a implémenté et pourquoi ça marche (ou pas)

2.1 Q-Learning tabulaire

Le Q-Learning stocke une table $Q[s][a]$ et met à jour chaque entrée avec la règle TD :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \underbrace{\left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]}_{\delta_t : \text{erreur TD}}$$

Dans notre implémentation, la table est un dictionnaire Python indexé par l'identifiant entier de l'état :

```
# Initialisation lazy : l'état est cree quand on le voit pour la
# premiere fois
if state_id not in self.q_table:
    self.q_table[state_id] = np.zeros(self.n_actions)

# Mise a jour TD avec action masking
td_error = reward + self.gamma * np.max(
    self.q_table[next_state_id][action_mask_next]
) - self.q_table[state_id][action]
self.q_table[state_id][action] += self.lr * td_error
```

L'action masking est critique ici — sur TicTacToe, jouer sur une case déjà occupée doit avoir une probabilité zéro. On masque les actions illégales en mettant leur Q-valeur à $-\infty$ lors de la sélection.

Limite fondamentale : cette table ne peut pas exister pour Quarto. Avec une observation de dimension 166, le nombre d'états distincts est astronomique — on ne peut même pas pré-allouer la mémoire.

2.2 La famille DQN : trois problèmes, trois solutions

2.2.1 DQN

DQN remplace la table par un réseau de neurones $Q_\theta(s) \in \mathbb{R}^{|A|}$ qui produit une Q-valeur pour chaque action simultanément. La mise à jour minimise la MSE entre la prédiction et la cible TD :

$$\mathcal{L}(\theta) = \underbrace{\left(r + \gamma \max_{a'} Q_\theta(s', a') - Q_\theta(s, a) \right)^2}_{\text{cible}}$$

```
# Calcul de la cible TD et de la perte
with torch.no_grad():
    next_q = self.q_net(next_state)
    next_q[~action_mask_next] = -float('inf')
    target = reward + self.gamma * next_q.max()

current_q = self.q_net(state)[action]
loss = F.mse_loss(current_q, target)
```

Observation expérimentale

DQN sur LineWorld : 100% en évaluation à 100k, mais la `policy_loss` moyenne est de **464 000** — une valeur qui confirme la divergence. En regardant l'entraînement par fenêtres de 500 épisodes, le win rate monte à 84% vers l'épisode 30k puis s'effondre à 20%. La performance finale de 100% est un coup de chance, pas de la stabilité.

Sur GridWorld : DQN ne converge pas du tout — 0% à tous les checkpoints. À 10k épisodes, les épisodes ont un `draw_rate` de 100% avec `steps = 10 000` (timeout) : l'agent tourne en boucle sans jamais atteindre le but.

Le problème est que sans target network, la cible $r + \gamma \max_{a'} Q_\theta(s', a')$ dépend de θ lui-même. Chaque mise à jour modifie à la fois la prédiction *et* la cible — le réseau "chasse ses propres cibles".

2.2.2 DDQN : résoudre la non-stationnarité des cibles

DDQN ajoute un **réseau cible** θ^- , copie de θ figée et synchronisée toutes les 100 steps. La cible devient :

$$y^{\text{DDQN}} = r + \gamma Q_{\theta^-} \left(s', \arg \max_{a'} Q_\theta(s', a') \right)$$

L'idée clé : on sépare la *sélection* de la meilleure action (réseau principal θ) et son *évaluation* (réseau cible θ^-). Ça réduit aussi le biais de surestimation de DQN — le max sur les Q-valeurs bruitées surestime toujours, mais si la sélection et l'évaluation sont décorréées, le biais est réduit.

```
# DDQN : selection avec theta, evaluation avec theta_minus
with torch.no_grad():
    # Selection : quelle action est la meilleure selon le reseau
    # principal ?
```

```
best_action = self.q_net(next_state).argmax()
# Evaluation : quelle est sa valeur selon le reseau cible ?
target_val = self.target_net(next_state)[best_action]
target = reward + self.gamma * target_val
```

Observation expérimentale

DDQN sans replay : 0% sur LineWorld et GridWorld à tous les checkpoints (loss_rate = 100%). Sur TicTacToe, la progression est régulière : 52.3% → 59.5% → **79.3%**. Le target network stabilise les cibles, mais sans décorrélation des transitions l'entraînement reste fragile sur les environnements simples où chaque épisode ne fait que 2 coups.

2.2.3 DDQN + Replay : décorréler les transitions

Le replay buffer stocke les transitions dans une mémoire circulaire de 50 000 éléments. À chaque step d'entraînement, on tire un mini-batch aléatoire :

```
class ReplayBuffer:
    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, mask, done):
        :
        self.buffer.append((state, action, reward, next_state,
                             mask, done))

    def sample(self, batch_size):
        return random.sample(self.buffer, batch_size)
```

En tirant aléatoirement dans 50 000 transitions accumulées, on brise la corrélation temporelle entre les mises à jour consécutives. Chaque transition peut aussi être réutilisée plusieurs fois, ce qui améliore l'efficacité d'échantillonnage.

2.2.4 DDQN + PER : toutes les transitions ne se valent pas

PER va plus loin : au lieu d'un tirage uniforme, on échantillonne les transitions proportionnellement à leur erreur TD absolue $|\delta_i|$:

$$P(i) = \frac{(|\delta_i| + \varepsilon)^\alpha}{\sum_j (|\delta_j| + \varepsilon)^\alpha}, \quad \alpha = 0.6$$

L'intuition : une transition avec $|\delta| \approx 0$ est déjà bien apprise — on perd du temps à la réapprendre. Une transition avec $|\delta|$ élevé est une erreur importante — l'agent a beaucoup à apprendre de celle-là.

Ce biais d'échantillonnage est corrigé par des poids d'importance sampling $w_i \propto (N \cdot P(i))^{-\beta}$ avec $\beta : 0.4 \rightarrow 1.0$ — on commence avec une correction partielle pour aller plus vite, et on converge vers une correction totale.

Pour que ça soit efficace (50 000 transitions, mise à jour et échantillonnage à chaque step), on utilise un **sum-tree** : un arbre binaire où chaque nœud parent contient la somme de ses enfants, permettant l'échantillonnage proportionnel en $\mathcal{O}(\log N)$.

Observation expérimentale

DDQN+PER converge à 100% dès 1k épisodes sur LineWorld et reste parfaitement stable jusqu'à 100k. La **policy_loss** est de **0.051** en moyenne, contre 464 000 pour DQN — un facteur 9 000. C'est la démonstration empirique que les trois mécanismes (target network + replay + PER) s'additionnent de manière cohérente.

2.3 REINFORCE et ses variantes de baseline

REINFORCE optimise directement la politique $\pi_\theta(a|s)$ en augmentant la probabilité des actions ayant conduit à un retour élevé :

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot (G_t - b(s_t))]$$

En pratique, on attend la fin de l'épisode, on calcule les retours Monte Carlo $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$, puis on fait une descente de gradient.

Le choix de la **baseline** $b(s_t)$ est central — elle réduit la variance sans biaiser le gradient. On a testé trois variantes :

Sans baseline $b = 0$. Implémentation minimale :

```
# Calcul des retours Monte Carlo
returns = []
G = 0
for r in reversed(rewards):
    G = r + self.gamma * G
    returns.insert(0, G)
returns = torch.tensor(returns)

# Gradient : on maximise log_prob * G
loss = -torch.stack(log_probs) * returns
```

Mean baseline $b = \bar{G}_{\text{épisode}}$. On soustrait simplement la moyenne des retours de l'épisode. Facile à implémenter, réduit la variance mais de manière grossière — la même valeur pour tous les états.

Critic baseline $b(s_t) = V_\phi(s_t)$, un réseau qui apprend la valeur d'état. L'avantage $A_t = G_t - V_\phi(s_t)$ mesure si l'action était *meilleure ou pire qu'attendu* depuis cet état spécifiquement :

```
# Le critique prédit V(s), la perte critic est MSE avec le retour
# Monte Carlo
value = self.critic(state).squeeze()
critic_loss = F.mse_loss(value, G_t)
```

```
# L'avantage utilise la prediction du critique comme baseline
advantage = G_t - value.detach()
actor_loss = -log_prob * advantage
```

Observation expérimentale

Sur TicTacToe à 1k épisodes : critic (69.8%) > PPO (70.8%) > no_baseline (61.3%) > mean (53.1%). Le critic aide clairement en début d'entraînement parce qu'il adapte la baseline à chaque état. Mais à 10k épisodes, toutes les variantes sont à 100% — l'environnement est trop simple pour que la réduction de variance reste décisive sur la durée.

Sur Quarto : toutes les variantes stagnent autour de 50%, y compris le critic. La variance n'est pas le seul problème ici.

2.4 PPO

PPO résout le problème de REINFORCE qui ne peut faire qu'une seule mise à jour par épisode. Avec REINFORCE, si on fait plusieurs passes de gradient sur le même batch, la politique change trop — on sort de la distribution des données, les gradients deviennent faux.

PPO contrainde chaque mise à jour à rester proche de la politique précédente via un ratio clipé :

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t A_t, \text{clip}(r_t, 1 - \varepsilon, 1 + \varepsilon) A_t)], \quad r_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

Le clip à $\varepsilon = 0.2$ signifie : si une action était bonne, on augmente sa probabilité, mais pas plus de $1.2\times$. Si elle était mauvaise, on la réduit, mais pas en dessous de $0.8\times$.

Grâce à ça, on peut faire **4 passes de gradient** sur le même rollout de 16 épisodes. PPO réutilise les données là où REINFORCE les jette après un seul gradient step.

```
# PPO : 4 epochs de gradient sur le meme rollout
for _ in range(self.update_epochs):
    new_log_probs, values = self.actor_critic(states)
    ratio = torch.exp(new_log_probs - old_log_probs.detach())

    # Objectif clippe
    surr1 = ratio * advantages
    surr2 = torch.clamp(ratio, 1 - self.clip_eps, 1 + self.
        clip_eps) * advantages
    actor_loss = -torch.min(surr1, surr2).mean()
    critic_loss = F.mse_loss(values.squeeze(), returns)

    loss = actor_loss + critic_loss
    loss.backward()
```

Observation expérimentale

Sur TicTacToe : PPO atteint 100% à 100k (70.8% → 99.7% → 100%), similaire aux variantes REINFORCE. L'environnement est trop simple pour révéler l'avantage des K epochs.

Sur Quarto : PPO stagne à 49.8% comme les autres méthodes model-free. Le clipping et les 4 epochs n'aident pas quand le problème fondamental est le manque de signal informatif, pas la variance des gradients.

2.5 MCRR et MCTS : planifier sans apprendre

Ces méthodes accèdent au modèle de l'environnement via `env.determinize()` pour simuler des parties complètes. Elles n'apprennent rien entre les parties — à chaque coup, elles raisonnent depuis zéro.

MCRR distribue son budget de simulations uniformément :

$$\hat{Q}(s, a) = \frac{1}{K} \sum_{k=1}^K R_k(s, a), \quad K = \left\lfloor \frac{N}{|\mathcal{A}_{\text{légal}}|} \right\rfloor$$

MCTS utilise la formule UCB pour concentrer les simulations sur les actions prometteuses :

$$\text{UCB}(v) = \frac{Q(v)}{N(v)} + c \sqrt{\frac{\ln N(\text{parent})}{N(v)}}, \quad c = 1.4$$

Observation expérimentale

MCTS (96.5%) > MCRR (94.6%) sur Quarto avec 100 simulations. L'avantage d'UCB est clair : avec 16 actions légales, MCRR fait seulement 6 simulations par action. MCTS peut en allouer 40+ aux coups prometteurs et 2 aux coups clairement mauvais.

Le coût : 35 ms/coup pour MCTS et 30 ms pour MCRR, contre 0.09 ms pour l'Expert Apprenti. Ratio de $\sim 390\times$.

2.6 Expert Apprenti

L'Expert Apprenti combine la qualité de la planification et la rapidité des réseaux appris. Pipeline en deux étapes :

1. **Collecte** : MCRR joue 10 000 épisodes (50 simulations/coup) et enregistre les Q-valeurs estimées pour chaque état visité.
2. **Distillation** : un réseau apprend à reproduire ces Q-valeurs par régression supervisée (MSE) sur 1 000 epochs.

```
# Phase 1 : collecte du dataset
for episode in range(num_episodes):
    state = env.reset()
    while not done:
        # Expert MCRR evalue chaque action par simulation
```

```

        q_values = mcrr.estimate_q_values(state, num_simulations
                                         =50)
        dataset.append((state, q_values))
        action = np.argmax(q_values[valid_actions])
        state, reward, done, _ = env.step(action)

# Phase 2 : entraînement supervise
for epoch in range(num_epochs):
    for batch_states, batch_q_targets in dataloader:
        pred = model(batch_states)
        loss = F.mse_loss(pred, batch_q_targets)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

```

La différence fondamentale avec REINFORCE/PPO c'est la richesse du signal : au lieu de $\{+1, -1\}$ terminal, le réseau apprend depuis $Q_{\text{MCRR}}(s, a) \in \mathbb{R}^{16}$ — 16 valeurs numériques qui encodent directement la qualité relative de chaque action dans cet état précis.

3 Résultats globaux

Table 1: Win rate au checkpoint 100k (ou meilleur epoch pour Expert Apprenti). Seed = 42, évaluation sur 1 000 épisodes.

Agent	LineWorld	GridWorld	TicTacToe	Quarto
Q-Learning	100.0%	100.0%	84.4%	N/A
DQN	100.0% [†]	0.0% ✗	74.7%	48.6%
DDQN	0.0% ✗	0.0% ✗	79.3%	48.1%
DDQN + Replay	100.0%	—	—	—
DDQN + PER	100.0%	—	—	—
REINFORCE (no baseline)	100.0%	100.0%	100.0%	48.8%
REINFORCE (mean baseline)	100.0%	100.0%	100.0%	49.0%
REINFORCE (critic baseline)	100.0%	100.0%	100.0%	52.6%
PPO	100.0%	100.0%	100.0%	49.8%
MCRR (100 sim)	100.0%	100.0%	87.7%	94.6%
MCTS (100 sim)	100.0%	100.0%	92.4%	96.5%
Expert Apprenti (ep. 1000)	100.0%	100.0%	87.8%	70.1%

[†] DQN LineWorld @100k : 100% en éval mais policy_loss = 464 000 en training (instabilité masquée).

✗ = agent qui n'apprend pas du tout.

4 Analyse par environnement

4.1 LineWorld et GridWorld : les révélateurs d'instabilité

Ces environnements sont triviaux — la politique optimale de LineWorld c'est littéralement "toujours aller à droite". Mais ils révèlent les instabilités algorithmiques que les

environnements complexes masqueraient.

DQN instable malgré une éval correcte Sur LineWorld, DQN affiche 100% en évaluation à 100k. Mais le suivi de l'entraînement (fenêtres de 500 épisodes) raconte une histoire différente :

Épisodes	0–5k	5k–30k	30k–45k	45k–70k	70k–100k
Win rate moyen	~50%	60–84%	20–40%	40–70%	50–72%

L'agent monte à 84% vers l'épisode 30k, s'effondre, puis oscille sans jamais converger. La `policy_loss` de 464000 confirme que la fonction de perte diverge. Sur GridWorld, DQN ne converge pas du tout (0% ou timeout à 10 000 steps).

L'explication : sans target network, la cible $r + \gamma \max_{a'} Q_{\theta}(s', a')$ dépend de θ lui-même. Chaque gradient step modifie simultanément la prédiction et la cible, créant une instabilité en boucle. Sans replay buffer, les 2 transitions consécutives d'un épisode LineWorld sont maximalement corrélées — les gradients sont biaisés.

DDQN sans replay : même symptôme 0% sur LineWorld et GridWorld (`loss_rate` = 100%). Le target network aide sur TicTacToe (52% → 79%) mais pas sur des environnements où les épisodes sont courts (2 coups), ce qui maximise la corrélation temporelle.

DDQN+PER : stabilité parfaite 100% dès 1k épisodes, `policy_loss` = 0.051, stable jusqu'à 100k. Les trois mécanismes (target network + replay + PER) s'additionnent de manière cohérente.

Longueur des épisodes sur GridWorld Parmi les agents qui réussissent, le nombre de pas moyen est instructif :

Agent	Steps moyens	Commentaire
Q-Learning, REINFORCE, PPO, Expert Apprenti	8.0	Chemin optimal mémorisé
MCTS (100 sim)	14.4	Arbre partiel, pas optimal
MCRR (100 sim)	20.8	Distribution uniforme inefficace

Les agents appris ont mémorisé le chemin minimal. Les méthodes de planification trouvent une solution mais pas la plus courte — elles ne mémorisent pas la politique entre les décisions.

4.2 TicTacToe : trois niveaux de performance

Table 2: Win rate sur TicTacToe — évolution par checkpoint.

Agent	@1k	@10k	@100k
Q-Learning	62.1%	80.2%	84.4%
DQN	77.4%	64.4%	74.7%
DDQN	52.3%	59.5%	79.3%
REINFORCE (no baseline)	61.3%	100.0%	100.0%
REINFORCE (mean baseline)	53.1%	100.0%	100.0%
REINFORCE (critic baseline)	69.8%	100.0%	100.0%
PPO	70.8%	99.7%	100.0%
MCTS (100 sim)		92.4%	
MCCR (100 sim)		87.7%	
Expert Apprenti (ep. 1000)		87.8%	

Niveau 1 : Policy-gradient à 100% REINFORCE et PPO convergent tous à 100% à 100k, mais la vitesse varie à 1k épisodes. REINFORCE critic (69.8%) et PPO (70.8%) devancent les autres parce que leur baseline ou leur clipping réduit la variance dès le début.

Ce qui est contre-intuitif : REINFORCE sans baseline (61.3%) n'est pas loin. Sur TicTacToe contre un adversaire aléatoire, le signal de victoire/défaite est suffisamment fréquent pour que même l'estimateur le plus bruyé converge en 10k épisodes. La mean baseline (53.1% à 1k) est paradoxalement la pire au départ — elle peut signer la variance dans la mauvaise direction si elle est mal calibrée en début d'entraînement.

Niveau 2 : Value-based instable à 75–79% DQN oscille : 77% → 64% → 75%. C'est la signature de l'instabilité sans target network sur un jeu adversarial — quand les Q-valeurs dévient, la politique change, l'adversaire aléatoire explore d'autres configurations, et ça peut désapprendre des stratégies acquises.

DDQN progresse régulièrement (52% → 59% → 79%) grâce au target network, mais plafonne à 79% sans replay buffer.

Q-Learning converge vers 84% de manière monotone et robuste (variance inter-seeds faible : 84.4%, 86.2%, 87.0%). Sa limite est structurelle : il ne peut pas généraliser entre deux configurations de plateau similaires — chaque état est une entrée indépendante dans la table.

Niveau 3 : Planification et imitation à 88–92% Sans entraînement, MCTS obtient 92.4% et MCCR 87.7%. L'Expert Apprenti (87.8%) atteint exactement le niveau de son professeur MCCR — on ne peut pas apprendre mieux que son expert par imitation pure.

4.3 Quarto : le mur du signal sparse

Table 3: Win rate sur Quarto — tous agents.

Agent	@1k	@10k	@100k / meilleur
DQN	44.7%	48.8%	48.6%
DDQN	49.9%	47.8%	48.1%
REINFORCE (no baseline)	48.9%	51.4%	48.8%
REINFORCE (mean baseline)	51.6%	48.3%	49.0%
REINFORCE (critic baseline)	49.2%	47.3%	52.6%
PPO	48.3%	50.2%	49.8%
Expert Apprenti (ep. 100)	—	—	58.1%
Expert Apprenti (ep. 500)	—	—	63.5%
Expert Apprenti (ep. 1000)	—	—	70.1%
MCCR (100 sim)	—	—	94.6%
MCTS (100 sim)	—	—	96.5%

Le résultat le plus frappant : **toutes les méthodes model-free stagnent autour de 50%** — statistiquement indistinguishable de l'aléatoire. Il n'y a aucune tendance à l'amélioration entre 1k et 100k épisodes. Les chiffres oscillent dans le bruit statistique ($\pm 3\%$ sur 1000 épisodes).

Pourquoi ça échoue ?

1. **Crédit assignment sur 2 phases** : en phase SELECT, l'agent choisit une pièce à donner à l'adversaire. L'effet de ce choix peut apparaître 4–6 tours plus tard. Avec $\gamma = 0.99$ et des épisodes de 14–26 coups, le gradient de $\nabla_{\theta} \log \pi(a_{\text{SELECT}}|s) \cdot G_t$ est extrêmement bruité.
2. **Espace immense, peu d'épisodes** : avec 16 pièces à 4 attributs, 16 positions, et 2 phases, on visite en 100k épisodes une fraction négligeable de l'espace d'état. Le réseau ne peut pas généraliser.
3. **Signal terminal seul** : $\{+1, -1\}$ après 14–26 coups. Rien n'indique si un coup intermédiaire était bon ou mauvais.

Pourquoi l'Expert Apprenti réussit ? 70.1% après 1000 epochs, avec une progression nette epoch 100 (58.1%) $\rightarrow$ epoch 500 (63.5%) $\rightarrow$ epoch 1000 (70.1%). La loss MSE décroît de 0.251 à 0.107 — le réseau apprend vraiment.

La différence est dans la nature du signal. L'Expert Apprenti ne reçoit pas un $\{+1, -1\}$ terminal. Pour chaque état, il reçoit **16 valeurs numériques** $Q_{\text{MCCR}}(s, a)$ estimées par 50 simulations chacune. Ces Q-valeurs encodent directement "dans cet état, l'action 3 est 1.8 fois meilleure que l'action 7". C'est un signal incomparablement plus riche.

La limite : le **distribution shift**. L'expert génère des données depuis ses propres trajectoires. Quand l'étudiant commet une erreur qui l'emmène dans un état non couvert par le dataset, sa politique devient imprédictible. C'est pour ça qu'on plafonne à 70% et pas à 95%.

Pourquoi MCTS/MCRR dominant ? 94–97% sans aucun entraînement, grâce à l'accès au modèle de l'environnement (`env.determinize()`). Là où les méthodes model-free *estiment* la valeur d'une action, MCTS/MCRR la *simulent* directement.

La différence MCTS (96.5%) vs MCRR (94.6%) : avec 16 actions légales et 100 simulations, MCRR fait 6 simulations par action en moyenne. MCTS peut en allouer 40 aux coups prometteurs et 2 aux coups clairement mauvais — c'est l'avantage d'UCB.

5 Analyse du coût computationnel

Table 4: Temps moyen par coup en inférence. Mesuré sur 1000 épisodes d'évaluation (seed = 42).

Agent	TicTacToe (ms)	Quarto (ms)	Ratio vs Expert
Q-Learning	0.07	N/A	<1×
Expert Apprenti	0.10	0.09	1×
DQN / REINFORCE / PPO	0.48–0.61	0.42–0.49	~5×
MCRR (100 sim)	13.6	30.3	~337×
MCTS (100 sim)	13.1	35.1	~390×

L'Expert Apprenti est le plus rapide parmi les méthodes profondes (0.09 ms/coup sur Quarto) — même plus rapide que REINFORCE et PPO qui ont des réseaux moins larges. Son réseau compact (3 couches × 128 neurones) n'a aucun rollout à calculer.

Le ratio MCTS/Expert Apprenti est de ~390× sur Quarto. Pour une contrainte de 10 ms/coup, MCTS est hors budget (35 ms) mais l'Expert Apprenti est largement en dessous (0.09 ms).

Rapport qualité/vitesse sur Quarto : l'Expert Apprenti (70.1%, 0.09 ms) domine clairement parmi les méthodes sans modèle de l'environnement. La distillation du planning dans un réseau rapide est la bonne approche pour la production.

6 Synthèse

6.1 Ce que les résultats confirment

La progression DQN → DDQN → DDQN+Replay → DDQN+PER se justifie empiriquement. L'écart de policy_loss (464 000 pour DQN vs 0.051 pour DDQN+PER, facteur 9 000) et les courbes d'entraînement montrent l'impact cumulatif de chaque mécanisme de stabilisation. On ne peut pas juste "mettre un réseau de neurones" et espérer que ça converge — il faut les deux stabilisateurs (target network + replay).

La baseline critic aide principalement en début d'entraînement. À 1k épisodes sur TicTacToe : critic (69.8%) > no_baseline (61.3%) > mean (53.1%). La baseline adaptée à chaque état $V_\phi(s)$ est strictement meilleure qu'une constante $\bar{G}$, surtout en début d'entraînement quand les retours varient beaucoup.

UCB est plus efficace que la distribution uniforme à budget égal. MCTS (96.5%) > MCRR (94.6%) sur Quarto, avec exactement 100 simulations. La concentration du budget sur les actions prometteuses vaut +1.9 points dans ce cas.

6.2 Les surprises

REINFORCE no _baseline converge aussi vite que PPO sur TicTacToe. On aurait attendu PPO nettement meilleur — mais les deux arrivent à 100% à 10k épisodes. L'environnement est trop simple pour que les 4 epochs de PPO fassent une différence.

DQN LineWorld : 100% en éval cache une divergence en training. Sans regarder la policy_loss (464 000) et les courbes d'entraînement, on aurait conclu que DQN fonctionne. C'est un cas où la métrique d'évaluation est trompeuse — l'agent a convergé par accident à 100k, pas par apprentissage stable.

L'Expert Apprenti creuse un écart de 17 points sur Quarto. 70.1% vs ~50% pour toutes les méthodes model-free. Ce n'est pas dû à une meilleure architecture ou un meilleur optimiseur — c'est uniquement la richesse du signal de supervision qui fait la différence.

6.3 Les limites

DDQN+Replay et DDQN+PER n'ont été testés que sur LineWorld — une évaluation sur TicTacToe et Quarto aurait été instructive. Un seul adversaire (Random) : un adversaire plus fort changerait la hiérarchie. 100k épisodes est insuffisant pour Quarto — les méthodes state-of-the-art utilisent des millions de parties avec du self-play.

7 Conclusion

Sur les environnements simples, le Q-Learning est optimal et imbattable en vitesse. Sur TicTacToe, les méthodes policy-gradient convergent à 100% là où le Q-Learning plafonne à 84% et les value-based instables à 75–79%. Sur Quarto, toutes les méthodes model-free échouent à apprendre quoi que ce soit en 100k épisodes, là où l'Expert Apprenti (70.1%) et MCTS (96.5%) dominent.

La leçon centrale : **la qualité du signal d'apprentissage prime sur la sophistication algorithmique.** REINFORCE avec $\{+1, -1\}$ terminal échoue là où l'Expert Apprenti avec des Q-values denses réussit sur Quarto. Et MCTS avec l'accès au modèle de l'environnement dépasse tout le monde sans jamais "apprendre". Choisir la bonne source de supervision est plus important que choisir entre REINFORCE et PPO.

References

- [1] Mnih et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518.
- [2] Van Hasselt, Guez, Silver (2016). Deep RL with double Q-learning. *AAAI*.
- [3] Schaul et al. (2016). Prioritized experience replay. *ICLR*.

- [4] Schulman et al. (2017). Proximal policy optimization. *arXiv:1707.06347*.
- [5] Williams (1992). REINFORCE. *Machine Learning*, 8.
- [6] Browne et al. (2012). A survey of MCTS methods. *IEEE TCIAIG*.